EC-341 Digital Systems Design
Experiment # 10

Analysis of Glitches and Hazards

Objectives:

- Glitches and Hazards
- Example
- Tasks

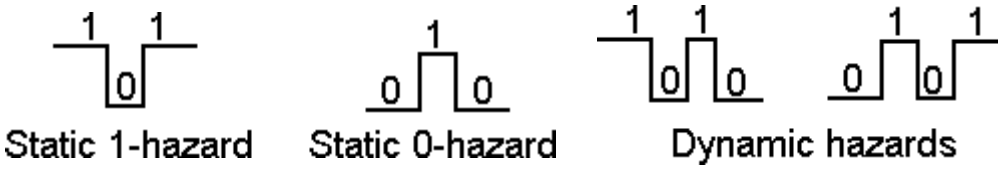
1. Introduction

A hazard is a characteristic of a circuit. A glitch is an unwanted pulse that may occur in a circuit with a hazard. A circuit with a hazard may or may not glitch. As an analogy a wet floor at the supermarket is a hazard. It's not a problem unless someone slips and falls which would be a glitch.

The state of a circuit and the pattern of input values determines if a circuit with a hazard will glitch.

Types of Hazards

Hazards may be static or dynamic. The 4 types of static and dynamic hazards are:



A static hazard is when the output should remain static but experiences an unwanted pulse. A dynamic hazard is when the output should go through a smooth transition but changes more than once before settling at the new value.

Our circuit had a static 0-hazard because the output should have stayed at 0 but momentarily went to 1.

Dealing with Hazards

Shortly we will discuss how to eliminate hazards but depending on the implementation a circuit with a hazard may not be a problem.

A circuit with a hazard isn't a problem if the output values are allowed to settle to a steady state before being sampled. Our example circuit above is asynchronous. Output values were allowed to change at any time and inputs were sampled at any time. We could have designed our circuit to be synchronous with the addition of a clock. Values in a synchronous circuit change according to a clock or reference signal. Values in the circuit are considered valid only at the beginning of a clock pulse. The time between the start and end of a clock pulse is enough to allow outputs to settle to a steady state.

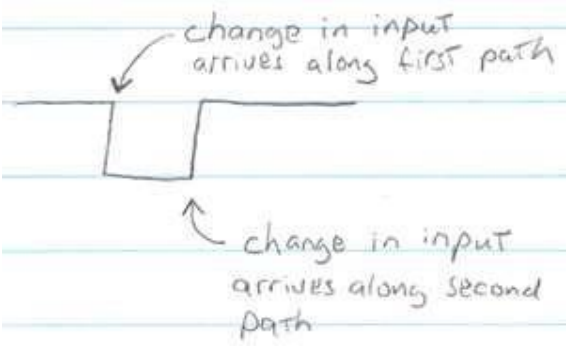
However, even in clocked synchronous circuits sometimes you have to deal with asynchronous inputs (for example, the D-type flip-flop we looked at earlier had asynchronous inputs for preset and clear.)

Detecting Hazards

Important note: we only discuss detecting and eliminating hazards that may cause a glitch during a single-variable change. It's much harder to detect and eliminate hazards that may cause a glitch during changes to more than one variable simultaneously.

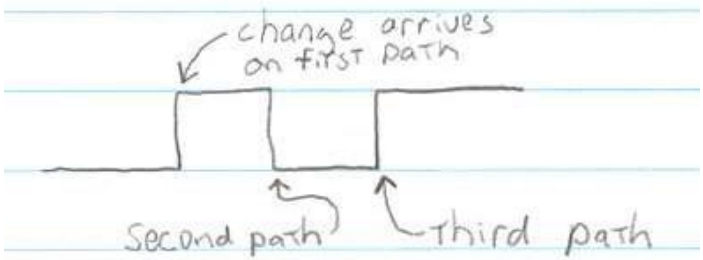
A necessary condition for a hazard is two or more paths with different propagational delays from one input variable that later converge.

Two paths with different propagational delays is sufficient to create a static hazard.



Change in one input variable arrives at the output at two different times causing a static-1 glitch

Three or more paths with different propagational delays are required to create a dynamic hazard:

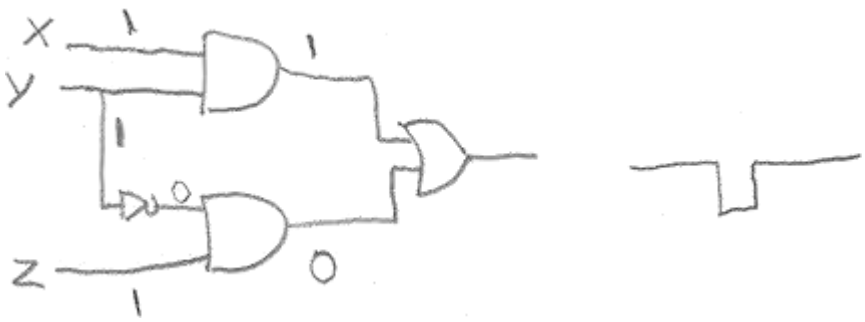


Change in one input variable arrives at the output at three different times causing a dynamic hazard

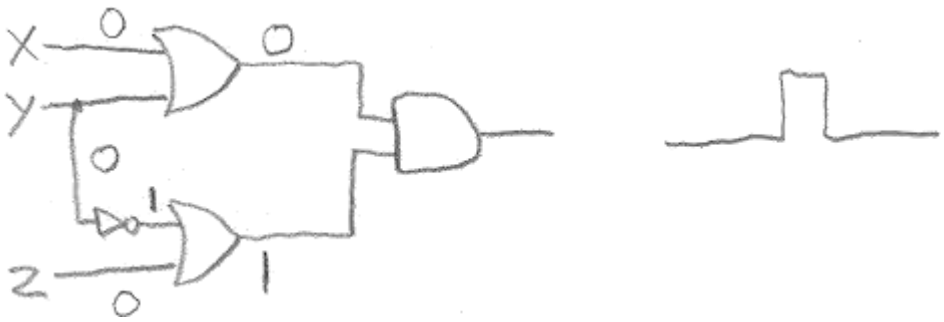
The more levels in a circuit the more opportunity there are for hazards. First let's consider hazards in two-level circuits.

If a circuit represents an equation that is in SOP or POS form that circuit has a hazard of the K-map representation of the circuit has implicants with adjacent ones (or zeros in the case of POS) that aren't covered by a common implicant.

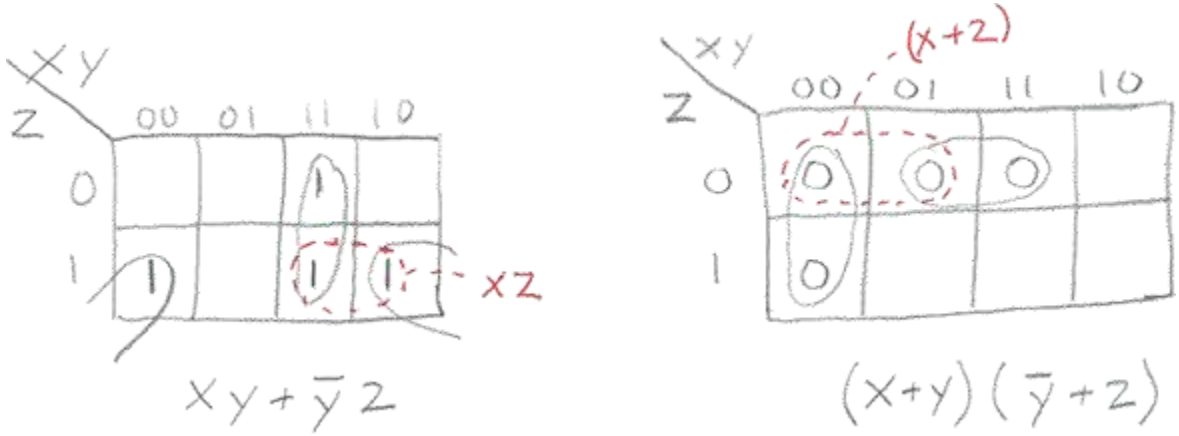
The SOP form of an expression is only susceptible to static 1-hazards.



The POS form of an expression is only susceptible to static 0-hazards.



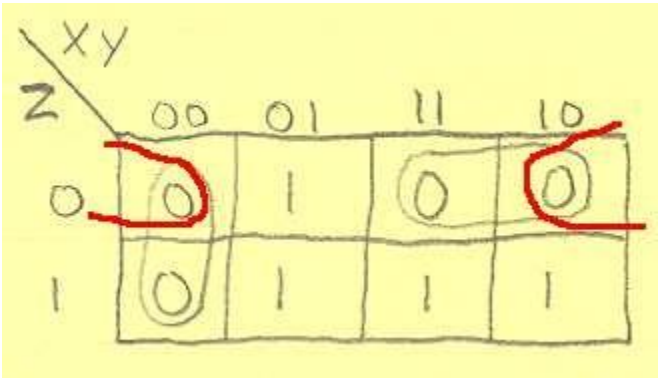
The problem is caused by a change in input arriving at the output of two different level 1 gates at two different times because of the delay of the inverter gate. Looking at the K-map for the above circuits reveals the solution.



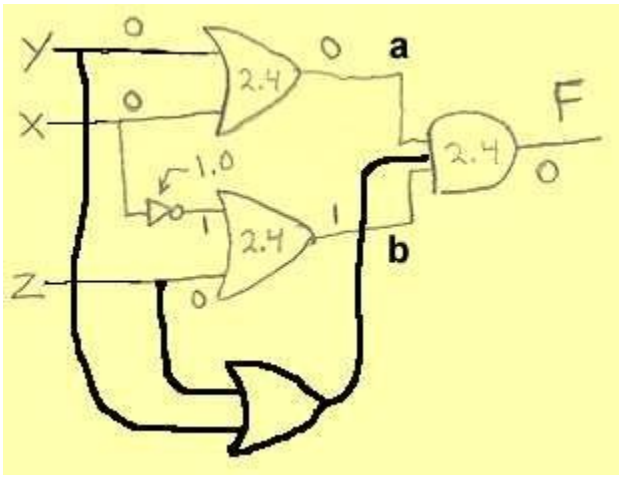
Prime implicants correspond to gates at the first level of the implementation. To eliminate static hazards from the standard form of an expression we need to add redundant prime implicants so that all adjacent minterms (or maxterms) are covered by an implicant. Now during any single bit change there will be a term that covers the transition. Or in other words there will be a term that doesn't change it's value during the transition. In the case of the SOP form of the expression this term will hold the output at 1. In the case of the POS form of the expression this term will hold the output at 0.

So, to eliminate static hazards from the standard form of an expression you include redundant prime implicants so that all adjacent minterms (or maxterms in the case of POS) are covered by a common product term (or sum term in the case of POS).

Example. Remove the glitch from the sample circuit given earlier which was (X + Y)(X' + Z). Adding redundant prime implicants to the K-map we get:



= (X + Y)(X' + Z)(Y + Z)

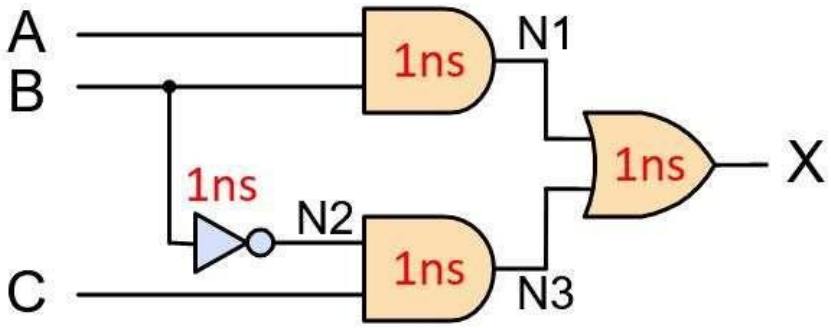


You can see from the diagram that the transition 000 --> 100 isn't a problem because the new term (Y + Z) stays constant during X's transition from 0 --> 1.

In summary, to detect and eliminate static hazards from a circuit:

I. Convert the circuit to it's two-level SOP or POS form.
II. Draw the K-map for the circuit
III. Adjacent 1's (or 0's if the circuit is in POS form) not covered by the same prime implicant indicate a static hazard exists.
IV. To remove the hazard add redundant prime implicants so that any two adjacent 1's (or 0's if the circuit is in POS form) are covered by the same prime implicant.

2. Example 1



Step 1: Implement the given circuit in Verilog

In the beginning, you are going to implement a circuit in Verilog and simulate it, taking delay into consideration. The circuit schematic in above figure, and the delay of each gate is marked in red.

The circuit shown in above figure has three inputs A, B, and C, and one output X. So, the declaration of the module goes as follows:

design.sv

```
1  // Code your design here
2
3  module CombCirc(A,B,C,X);
4
5    input A,B,C;
6    output X;
7
8    // Circuit Discription
9
10 endmodule
```

The goal of this example is to simulate the delay of logic gates and analyze its effect on the circuit behavior. So, you need to describe each logic gate use an assign statement with delay command as shown in the code block below.

```
wire N1, N2, N3;

assign #1 N1 = A & B;
assign #1 N2 = ~B;
assign #1 N3 = N2 & C;
assign #1 X = N1 | N3;
```

In the codes above, N1, N2 and N3 are three internal signals as labeled in given figure. Each gate has a delay of 1 reference time unit, in order to make the delay 1ns for each gate.

So the Verilog file that describes the circuit with delay information of each gate looks as follows:

design.sv

```
1  // Code your design here
2
3  module CombCirc(A,B,C,X);
4
5    input A,B,C;
6    output X;
7
8    wire N1, N2, N3;
9
10   assign #1 N1 = A & B;
11   assign #1 N2 = ~B;
12   assign #1 N3 = N2 & C;
13   assign #1 X = N1 | N3;
14
15 endmodule
```

Step 2: Create the test bench

In order to simulate glitch, you will need to generate an input sequence in your test bench that can cause the glitch to appear at the output of the circuit. By observing the circuit, there is an unbalanced path between input B and output X (i.e., there are two paths to propagate the changes of B to the output with different delays). So, the glitch will happen when “A” and “C” are held constant and B is toggled. The codes below generate a sequence where B changes from 1 to 0 and back to 1 when “A” and “C” are held constant. Do not forget to instantiate the “CombCirc” module in your test bench and connect the inputs and outputs to internal signals!

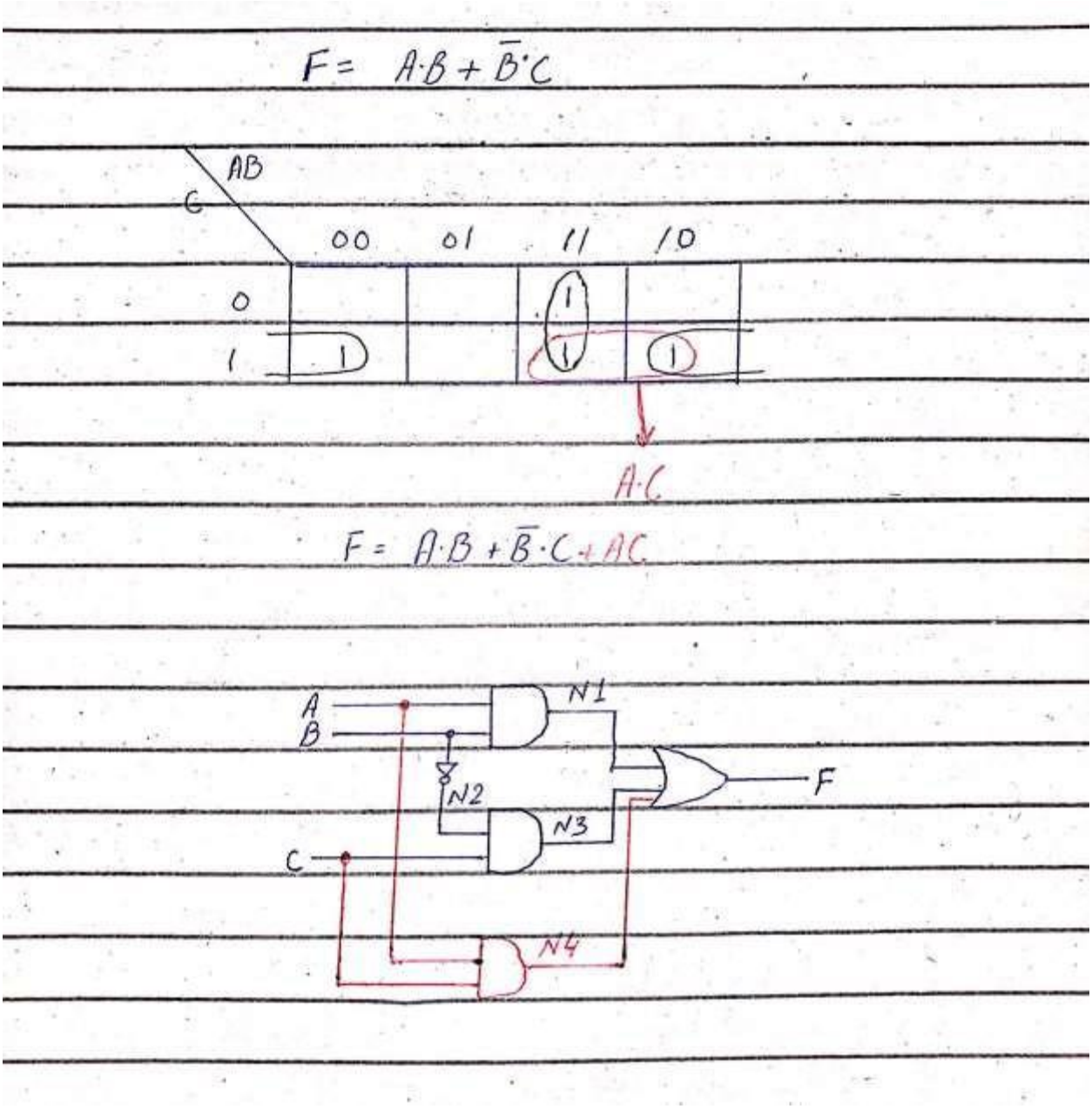
testbench.sv

```
// Code your testbench here
// or browse Examples

module CombCirc_TB ();

  integer k = 0;
  reg A,B,C;
  wire X;
  CombCirc uut(A,B,C,X);
  initial
    begin
      $dumpfile("dump.vcd");
      $dumpvars(1);
      A = 0;
      B = 0;
      C = 0;
      for(k = 0; k < 4; k=k+1)
        begin
          {A,C} = k;
          #5 B = 1;
          #5 B = 0;
          #5 ;
        end
    end
endmodule
```

Step 3: Simulate the test bench

Now, you can simulate your test bench in the Vivado/ModelSim/online simulator, and you will get the waveform as shown in figure below. The red circle on the waveform specifies the glitch. It happens when A is 1, C is 1 and B changes from 1 to 0. The duration of the glitch is 1ns.

Step 4: Elimination of Glitch



Step 1: Implement the given circuit in Verilog

design.sv

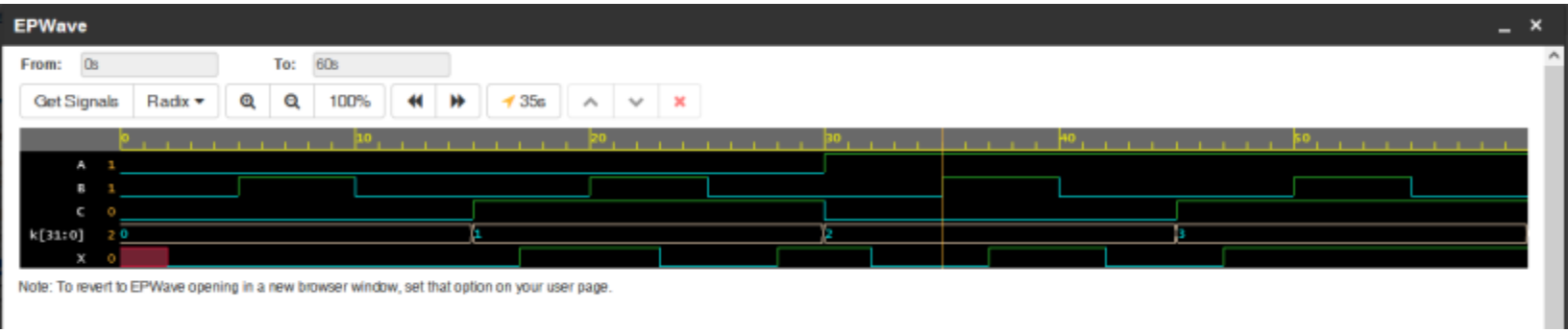
```
// Code your design here

module CombCirc(A,B,C,X);

  input A,B,C;
  output X;

  wire N1, N2, N3, N4;

  assign #1 N1 = A & B;
  assign #1 N2 = ~B;
  assign #1 N3 = N2 & C;
  assign #1 N4 = A & C;
  assign #1 X = N1 | N3 | N4;

endmodule
```

Step 2: Create the test bench & simulate the circuit

testbench.sv

```
// Code your testbench here
// or browse Examples

module CombCirc_TB ();

  integer k = 0;
  reg A,B,C;
  wire X;
  CombCirc uut(A,B,C,X);
  initial
    begin
      $dumpfile("dump.vcd");
      $dumpvars(1);
      A = 0;
      B = 0;
      C = 0;
      for(k = 0; k < 4; k=k+1)
        begin
          {A,C} = k;
          #5 B = 1;
          #5 B = 0;
          #5 ;
        end
    end
endmodule
```

Glitches Free Circuit: -

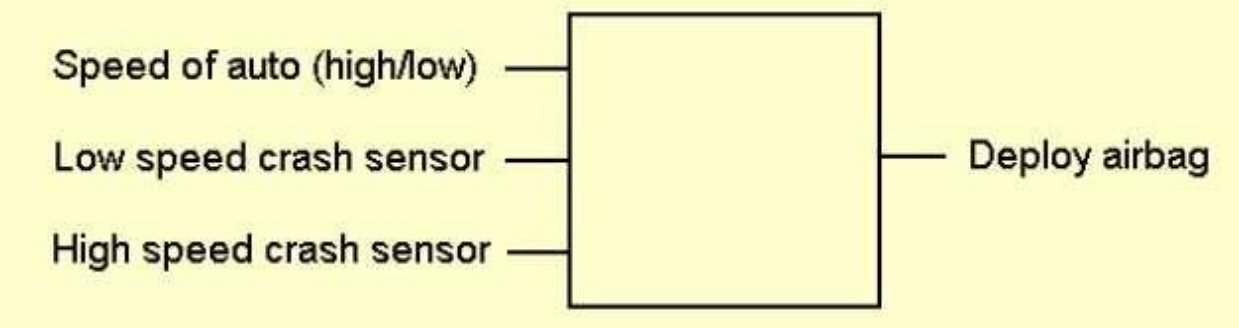


3. Example 2 (Air Bag Deployment)

Hazards and glitches will be introduced by way of an example.

You have been asked to design the combinational logic circuit for a safer airbag. What makes first generation airbags unsafe is that they may go off during a light accident and cause more damage to the occupants of the car than the accident would have caused. To better predict when the airbag should be deployed the safety engineers designed two sensors: one high speed sensor and one low speed sensor. When the car is traveling slow (<= 30mph) the slow speed sensor should determine if the airbag should be deployed. When the car is traveling fast (> 30mph) the high speed sensor should determine if the airbag should be deployed. Each sensor may give a false reading when the car is traveling outside of the speed range for which it was designed.

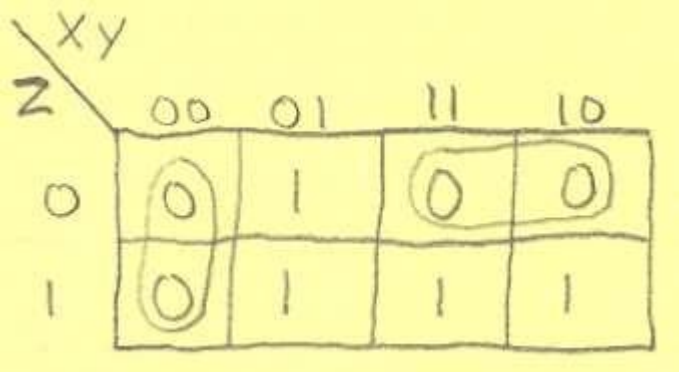
Your circuit will have 3 inputs and one output:



The next step is to create a truth table that defines the function of the circuit:

Speed (0 when <=30) X	Low speed sensor Y	High speed sensor Z	Deploy F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

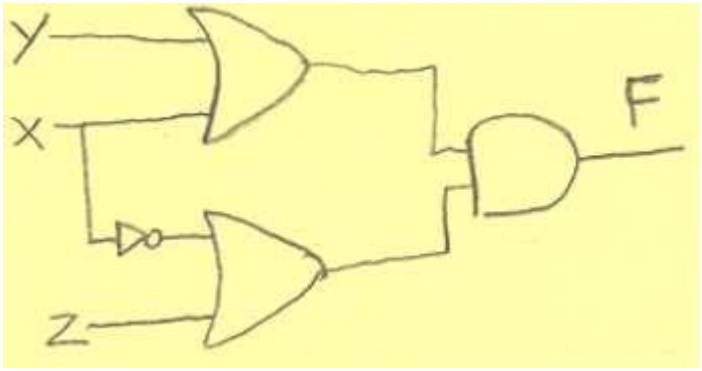
We can use a K-map to find the minimum two-level form of the expression:



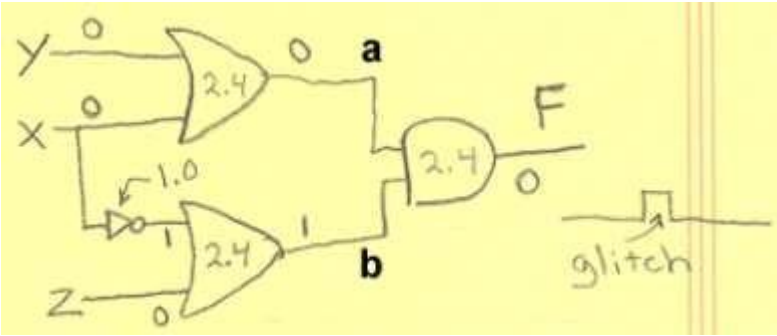
The minimized POS expression is: (X + Y)(X' + Z).

(Note, we choose to use the POS rather than the SOP form of the expression. You may notice that if we used the SOP form of the expression this example would be trouble free. That doesn't imply that SOP is always trouble-free. For example, if the "Deploy Airbag" signal is active low and the SOP form of the expression is used with the new truth table, you will see that the problem reappears.)

The circuit schematic then looks like:

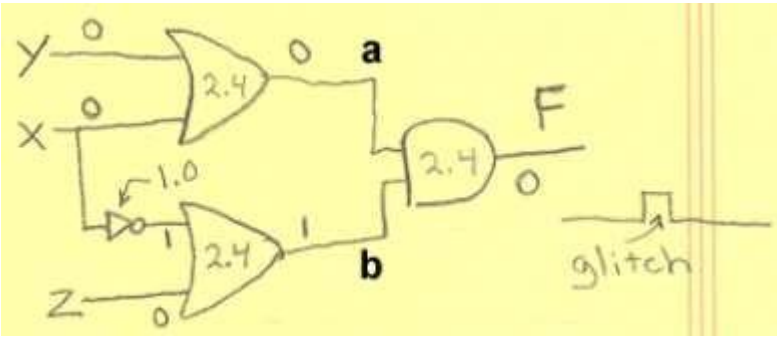


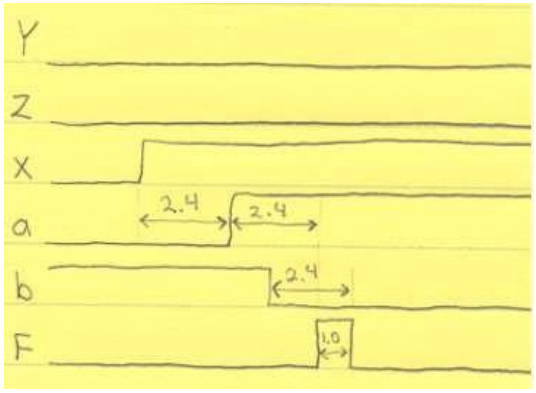
After double checking our design for errors we implement our solution in a test vehicle. The test engineer gets into the car to drive it to the crash testing facility that is just down the road. He pulls away from the curb and starts to accelerate. Just when the car reaches 31 mph--bam! he gets hit in the face with the airbag. What was wrong with the design? In short a **static hazard** in the circuit caused an unwanted pulse or **glitch** on the output that caused the airbag to deploy. To see how this could happen consider the figure below.



Notice that the propagational delay of each gate has been added to the circuit schematic. When X (speed of the car) goes from 0 (<= 30 mph) to 1 (> 30 mph) this change in X reaches F before the complement of X because of the added delay of the inverter gate.

More precisely the following timing diagram shows the dynamic behavior of the circuit including the 1 nano second glitch in the output.


Y
X
Z
0
0
0
2.4
a
1.0
1
2.4
1
b
2.4
F
0
glitch


Y
Z
X
a
2.4
2.4
b
2.4
F
1.0

4. Tasks

1. Write Verilog code for the above example in which first you must bring up a glitch. Than try to remove the glitch by making necessary changes in the code as well as the design. Run the stimulus (testbench) and show waveforms for both the designs.